

Motic®



MUCam SDK

Quick Start Guide

Contents

INTRODUCTION	- 1 -
OBJECTIVE	- 2 -
USING THE MUCAM API	- 2 -
FINDING CAMERAS	- 2 -
OPENING A CAMERA	- 3 -
BINNING AND ROI	- 4 -
FRAME FORMAT AND BIT DEPTH	- 4 -
GRABBING FRAMES	- 5 -
EXPOSURE, GAIN AND OFFSET	- 6 -
COOLER	- 7 -
MUCAM API FUNCTIONS	- 8 -
TYPE DEFINITION	- 8 -
typedef void* MUCam_Handle	- 8 -
ENUMERATIONS	- 8 -
enum MUCam_Type	- 8 -
enum MUCam_Format	- 14 -
enum MUCam_Binning_Type	- 14 -
enum MUCam_Trigger_Type	- 15 -
FUNCTIONS	- 16 -
MUCam_Handle MUCam_findCamera	- 16 -
void MUCam_releaseCamera	- 16 -
MUCam_Type MUCam_getType	- 17 -
bool MUCam_openCamera	- 17 -
void MUCam_closeCamera	- 18 -
bool MUCam_isConnected	- 18 -
MUCam_Format MUCam_getFrameFormat	- 19 -
bool MUCam_getFrame	- 19 -

bool MUCam_setBitCount	- 20 -
int MUCam_getBinningCount	- 20 -
bool MUCam_getBinningList	- 21 -
bool MUCam_getBinningType	- 21 -
bool MUCam_setBinningIndex	- 22 -
bool MUCam_setROI	- 22 -
bool MUCam_setFlip	- 23 -
bool MUCam_setMirror	- 23 -
int MUCam_getGainCount	- 24 -
bool MUCam_getGainList	- 24 -
bool MUCam_setRGBGainIndex	- 25 -
bool MUCam_setRGBGainValue	- 25 -
bool MUCam_getOffsetRange	- 27 -
bool MUCam_setRGBOffset	- 28 -
bool MUCam_getExposureRange	- 29 -
bool MUCam_setExposure	- 29 -
bool MUCam_isCoolerAvailable	- 30 -
bool MUCam_activateCooler	- 30 -
int MUCam_getFrequencyCount	- 30 -
int MUCam_getFrequencyIndex	- 31 -
bool MUCam_setFrequencyIndex	- 31 -
bool MUCam_setTriggerType	- 32 -
bool MUCam_getTemperature	- 32 -
bool MUCam_isUSB3Cam	- 32 -
int MUCam_getBayerFormat	- 33 -
bool MUCam_getBayer	- 33 -
bool MUCam_bayer2RGB	- 34 -
bool MUCam_bayer2RGB	- 35 -
bool MUCam_bayer2RGB	- 35 -
bool MUCam_bayer2RGB	- 36 -
bool MUCam_bayer2RGB	- 37 -

bool MUCam_setAutoExposure	- 37 -
bool MUCam_setAERect	- 38 -
bool MUCam_setWhiteBalance	- 38 -
bool MUCam_setAWBRect	- 39 -

Introduction

This document is a programmer's guide to the Motic Universal Camera (MUCam) API, which is the programming interface to various Motic USB cameras.

On Windows the MUCam SDK(Software Development Kit) includes MoticUCam.h, MUCam32.dll/lib files.

The different layers of interaction between an application, MUCam API and the operating system are detailed in the following illustration:

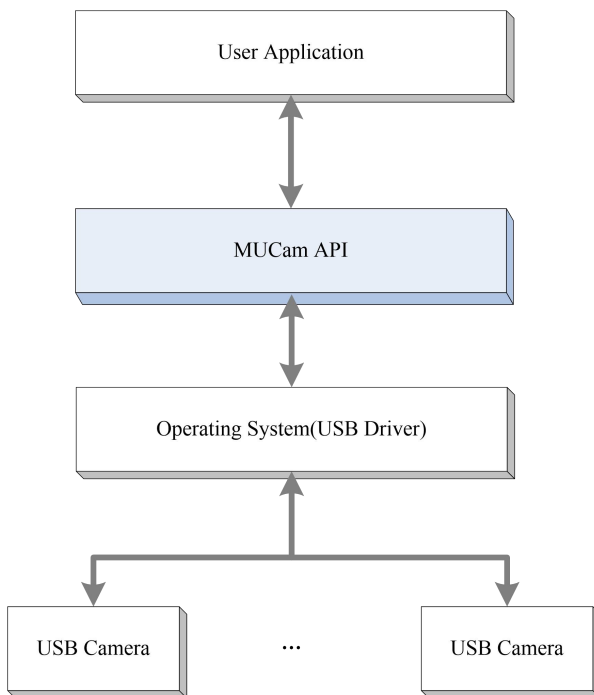


Fig-1 MUCam Layer Structure



Objective

The object of this document is to illustrate how to use the MUCam API as well as provide documentation for each available function in the API. The reader of this document should have a good knowledge of C/C++.

Using the MUCam API

The following sections describe how to use the API to do various operations such as finding and opening a camera, grabbing an image and getting/setting camera properties.

Note: The examples shown do not contain error checking. It has been omitted for the sake of clarity.

Finding Cameras

Before using a camera, find the camera handle with `MUCam_findCamera`; after using a camera, release the camera resources with `MUCam_releaseCamera`.

```
//to find a camera supported by MUCam
MUCam_Handle hCameraList[10];
MUCam_Handle hCameraHandle = MUCam_findCamera();
int indx = 0;
while(hCameraHandle)
{
    //add the camera to the camera list.
```



```
hCameraList[indx++] = hCameraHandle;

//get the camera type to determine camera name.
MUCam_Type type = MUCam_getType(hCameraHandle)
//...

//find the next camera
hCameraHandle = MUCam_findCamera();
} //end while

//using the cameras
//...

//release all resources associated with a camera
for(int i = 0; i < indx; i++)
{
    MUCam_releaseCamera(m_hCameraList[i]);
    hCameraList[i] = 0;
}
```

Opening a Camera

Select a camera with the camera handle found above, and open it with `MUCam_openCamera`; close the camera with `MUCam_closeCamera` after using it.

```
hCameraHandle = m_hCameraList[0];
if(MUCam_openCamera(hCameraHandle))
{
    //camera settings and grab frame
    //...
    //close the camera
    MUCam_closeCamera(hCameraHandle);
}
```



Binning and ROI

After opening a camera, you can get and set the resolution supported by the camera, or define a region of interest (ROI) to display a target image size.

```
//get binning count
int binCount = MUCam_getBinningCount(hCameraHandle);
int* m_pWidth = new int[m_iBinningCount];
int* m_pHeight = new int[m_iBinningCount];
//get the binning list
MUCam_getBinningList(hCameraHandle, m_pWidth, m_pHeight);
//by default we select binning 0 to display
int binningIndx = 0;
MUCam_setBinningIndex(hCameraHandle, binningIndx);
//...

//set ROI to display an ROI image
int top, left, bottom, right;
//GetROIRect(&top, &left, &bottom, &right);
MUCam_setROI(hCameraHandle, &top, &left, &bottom, &right);
```

Frame Format and Bit Depth

After opening a camera, you may get image format or modify image bit depth if needed.

```
//by default we set the image bit depth 8 bits
int bitCount = 8;
if(MUCam_setBitCount(hCameraHandle, bitCount) == false)
{
    //current bit depth is not supported.
}
//end if
//frame format
```



```
int colorChannel = 3;//default 3 channels(B,G,R)
MUCam_Format fmt = MUCam_getFrameFormat(hCameraHandle);
if(fmt == MUCAM_FORMAT_MONOCHROME)
{
    //monochrome camera
    colorChannel = 1;//only 1 channel
}
```

Grabbing Frames

After opening a camera, you can get image data. When a camera is opened, an interior thread will activate to grab images incessantly from the camera, and the image data will be stored in the interior buffer in time order. Use `MUCam_getFrame` to get image data; if failed to get data many times, invoke `MUCam_isConnected` to check whether the camera is connected.

```
//prepare the buffer, must be large enough to hold the image data
int width = m_pWidth[0];
int height = m_pHeight[0];
int bytes = 1;
if(bitCount <=8)
{
    bytes = 1;
}
if(bitCount > 8 && bitCount <= 16)
{
    bytes = 2;
}
unsigned char*buf = new unsigned char[width*height*bytes* colorChannel];
int failTimes = 0;
while(isRunning)
{
    if(MUCam_getFrame (hCameraHandle, buf, 0))
    {
        //the image can be a monochrome or a 16-bit image,
        //conversion may be needed to display the image properly.
    }
}
```



```
    Conver_and_Display(buf);
} //end if
else
{
    if(failTimes ++ > 5)
    {
        failTimes = 0;
        if(MUCam_isConnected(hCameraHandle) == false)
        {
            //the camera is unconnected
            break;
        }
    }
    Sleep(100);
} //end else
} //end while
//release buffer and resource
//...
```

Exposure, Gain and Offset

After opening a camera, you may get and set different parameters. The range of exposure may alter after modifying Binning or ROI, you should check it.

```
//exposure
float minExposure;
float maxExposure;
int minOffset;
int maxOffset;
int gainCount
float* pGain;
//get exposure range
MUCam_getExposureRange(hCameraHandle, &minExposure, &maxExposure);

//get Offset range
MUCam_getOffsetRange(hCameraHandle, &minOffset, &maxOffset);
```



```
//get Gain count
gainCount = MUCam_getGainCount(hCameraHandle);
//get Gain list
m_pGain = new float[m_gainCount];
MUCam_getGainList(hCameraHandle, m_pGain);

//set the exposure value, by default we set exposure time = 10ms
float fval = 10.0f; //[minExposure, maxExposure]
MUCam_setExposure(hCamera, fval);

//set the offset value, by default we set offset value = 0
int val = 0; //[minOffset, maxOffset]
MUCam_setRGBOffset(hCameraHandle, val, val, val);

//set the RGBGain, by default we set gain value = 1
int rindx, gindx, bindx;
MUCam_setRGBGainValue(hCameraHandle, 1.0f, 1.0f, 1.0f,
                      &rindx, &gindx, &bindx);
```

Cooler

Not all cameras support cooler function. Please check whether a camera's cooler is available before activate the function. Some camera's cooler needs to be connected to an external power supply. If not, the cooler cannot be activated.

```
if(MUCam_isCoolerAvailable(hCameraHandle))
{
    if(!MUCam_activateCooler(hCameraHandle))
    {
        //the cooler is available but cannot be activated.
    }
}
```



MUCam API Functions

Type Definition

- `typedef void* MUCam_Handle`

The camera object handle type. The handle is the "void *" type, never convert it to "int" or "long", especially in 64-bit operating system.

Enumerations

- `enum MUCam_Type`

The camera identifier, used to determine camera name in an application.

Enumerator:

<i>MUCAM_TYPE_UNKNOWN</i>	Unknown camera type.
<i>MUCAM_TYPE_MC1001</i>	MC1001 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 15(1280*1024) or 40(640*512).
<i>MUCAM_TYPE_MC2001</i>	MC2001 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 10(1600*1200) or 30(800*600).



<i>MUCAM_TYPE_MC3001</i>	MC3001 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 6(2048*1536) or 21(1024*768).
<i>MUCAM_TYPE_MC2001B</i>	MC2001B camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 10(1280*1024) or 30(640*512).
<i>MUCAM_TYPE_MC1002</i>	MC1002 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 14(1280*1024) or 52(640*512).
<i>MUCAM_TYPE_MC2002</i>	MC2002 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 5(1600*1200) or 20(800*600).
<i>MUCAM_TYPE_MA205</i>	MA205 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 10(1360*1024).
<i>MUCAM_TYPE_MA285</i>	MA285 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 15(1360*1024).
<i>MUCAM_TYPE_MA252</i>	MA252 camera type. On Windows it exports BGR format images. On MacOS



	X it exports RGB format images. The normal FPS is 11(2048*1536).
<i>MUCAM_TYPE_SWIFT_MC1002</i>	Swift MC1002 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 14(1280*1024) or 52(640*512).
<i>MUCAM_TYPE_MD3300</i>	Undocumented. Deprecated: Since MA252 added.
<i>MUCAM_TYPE_MC5001</i>	MC5001 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 7(2592*1944) or 20(1296*972).
<i>MUCAM_TYPE_MC3111</i>	MC3111 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images the normal FPS is 13(1280*1024) or 40(640*480)
<i>MUCAM_TYPE_MC3222</i>	MC3222 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images the normal FPS is 10(1600*1200) or 30(800*600)
<i>MUCAM_TYPE_MC3022</i>	MC3022 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. the normal FPS is 7(1600*1200) or 20(800*600)



<i>MUCAM_TYPE_SWIFT_MC3001</i>	Swift MC3001 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 6(2048*1536) or 21(1024*768).
<i>MUCAM_TYPE_SWIFT_MC3222</i>	Swift MC3222 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. the normal FPS is 10(1600*1200) or 30(800*600)
<i>MUCAM_TYPE_SWIFT_MC3111</i>	Swift MC3111 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images the normal FPS is 13(1280*1024) or 40(640*480)
<i>MUCAM_TYPE_MA282</i>	MA282 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 6(2560*1920).
<i>MUCAM_TYPE_MC10M</i>	MC10M camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 2.2(3664*2748).
<i>MUCAM_TYPE_IMX224</i>	MoticamS1 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 120(1280*960) or 240(640*480).
<i>MUCAM_TYPE_IMX178</i>	MoticamS6 camera type. On Windows it exports BGR format images. On MacOS



	X it exports RGB format images. The normal FPS is 30(3072*2048) or 50(1536*1024)
<i>MUCAM_TYPE_IMX123</i>	MoticamS3 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 60(2048*1536) or 60(1920*1080)
<i>MUCAM_TYPE_IMX226</i>	MoticamS12 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 25(4000*3000) or 50(2048*1080)
<i>MUCAM_TYPE_IMX250</i>	MoticamProS5 Plus camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 68 (2448*2048) or 175(1224*1024)
<i>MUCAM_TYPE_IMX264</i>	MoticamProS5 Lite camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 35(2448*2048) or 88(1224*1024)
<i>MUCAM_TYPE_IMX183</i>	MoticamS20 camera type. On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 18(5472*3648) or 20(2736*1824)
<i>MUCAM_TYPE_IMX253</i>	MoticamProS12 camera type. On Windows it exports BGR format images.



	On MacOS X it exports RGB format images. The normal FPS is 29(4000* 3000) or 91(2000*1500)
<i>MUCAM_TYPE_IMX250M</i>	MoticamProS5 Plus M camera type. Monochrome image format. The maximum FPS is 68 (2448* 2048) or 175(1224*1024)
<i>MUCAM_TYPE_IMX264M</i>	MoticamProS5 Lite M camera type. Monochrome image format. The maximum FPS is 35(2448* 2048) or 88(1224*1024)
<i>MUCAM_TYPE_IMX178M</i>	MoticamS6 M camera type. Monochrome image format. The maximum FPS is 30(3072*2048) or 50(1536*1024)
<i>MUCAM_TYPE_IMX253M</i>	MoticamProS12 M camera type. Monochrome image format. The maximum FPS is 29(4000* 3000) or 91(2000*1500)
<i>MUCAM_TYPE_IMX546</i>	MoticamProS8 Lite On Windows it exports BGR format images. On MacOS X it exports RGB format images. The normal FPS is 29(4000* 3000) or 91(2000*1500)
<i>MUCAM_TYPE_IMX546M</i>	MoticamProS8 Lite M camera type. Monochrome image format. The maximum FPS is 29(2816* 2816) 或 91(1408*1408)



- enum MUCam_Format

The camera exported image format definition.

Enumerator:

<i>MUCAM_FORMAT_BAYER_GR_BG</i>	The bayer image format in GRBG pattern.
<i>MUCAM_FORMAT_BAYER_BG_GR</i>	The bayer image format in BGGR pattern.
<i>MUCAM_FORMAT_BAYER_GB_RG</i>	The bayer image format in GBRG pattern.
<i>MUCAM_FORMAT_BAYER_RG_GB</i>	The bayer image format in RGGG pattern.
<i>MUCAM_FORMAT_COLOR_RGB</i>	The RGB color image format.
<i>MUCAM_FORMAT_COLOR_BGR</i>	The BGR color image format.
<i>MUCAM_FORMAT_MONOCHROME</i>	The monochrome image format.

- enum MUCam_Binning_Type

The camera binning type.

Enumerator:

<i>MUCAM_BINNING_NORMAL</i>	Normal binning type.
-----------------------------	----------------------



MUCAM_BINNING_SAMPLING

The sampling type completed by software.

MUCAM_BINNING_FAST_DISPLAY

The fast display type for high-resolution camera.

- **enum MUCam_Trigger_Type**

The camera grab frame type.

Enumerator:

MUCAM_TRIGGER_FREE

The trigger free mode. The SDK will automatically grab frames as soon as possible. It's the default mode when a camera opened.

MUCAM_TRIGGER_SOFTWARE

The software trigger mode. Every time invoking MUCam_getFrame function will cause camera to exposure and transfer a frame data.

MUCAM_TRIGGER_HARDWARE_RISE

The external hardware trigger signal on TTL rise side.

GMUCAM_TRIGGER_HARDWARE_FALL

The external hardware trigger signal on TTL fall side.



Functions

- [MUCam_Handle](#) MUCam_findCamera()

To find a camera supported by MUCam. This function will search for an unused camera. If find a camera then return its handle, otherwise return 0. The returned camera object is uninitialized. It should be released by MUCam_releaseCamera function after use. The search order is not fixed. Continuously invoke this function until it returns 0 to find all connected cameras.

Returns:

The camera handle if successful, 0 if failed.

See also:

[MUCam_openCamera](#), [MUCam_releaseCamera](#)

- void MUCam_releaseCamera([MUCam_Handle](#) camera)

To release a camera object. It is unnecessary to close the camera before releasing it.

Parameters:

camera The camera handle, not valid after return.

See also:



MUCam_findCamera

- [MUCam_Type](#) MUCam_getType([MUCam_Handle](#) camera)

To get the camera identifier. The function can be invoked without opening the camera.

Parameters:

camera The camera handle.

Returns:

The camera identifier.

See also:

[MUCam_Type](#)

- `bool` MUCam_openCamera([MUCam_Handle](#) camera)

To open the camera. The camera should be opened before using it. The camera status after opening is uncertain, the application should set all camera status parameters before capturing an image. The function will return "true" when it is invoked to open an opened camera.

Parameters:

camera The camera handle.

Returns:

True if successful; False if failed.

See also:

MUCam_closeCamera

- void MUCam_closeCamera([MUCam_Handle camera](#))

To close the opened camera. The closed camera can be re-opened. The MUCam_releaseCamera function should be invoked to release all resources associated with a camera.

Parameters:

camera The camera handle.

see also: [MUCam_releaseCamera](#)

- bool MUCam_isConnected([MUCam_Handle camera](#))

To check whether the camera is still connected. Invoke this function to check whether the camera has been un-plugged if failed to get images from camera many times.

Parameters:

camera The camera handle.

Returns:

True if camera is connected; False if camera has been removed.



- [MUCam_Format](#) `MUCam_getFrameFormat(MUCam_Handle camera)`

To get the image format exported by the camera.

Parameters:

camera The camera handle.

Returns:

The image format.

- `bool MUCam_getFrame (MUCam_Handle camera, unsigned char * buf, unsigned long * ts)`

To grab a frame

Parameters:

camera The camera handle.

buf The frame buffer, must be large enough to hold the image data.

ts The pointer to the buffer that will receive time stamp(ms) of the frame. Might be 0 when it is not used.

Returns:

True if successful; False if failed.

See also:

[MUCam_getFrameFormat](#)



- `bool MUCam_setBitCount(MUCam_Handle camera, int bit)`

Set bit depth of image data.

After setting bit depth, the captured images will be stored according to the bit depth. The processing of data byte order is fulfilled by MUCam automatically. The image data acquired by the application will be within the range defined by bit depth.

Parameters:

camera The camera handle

bit Bit depth, e.g.

- 8 means 8 bits per channel, stored in 1 byte.
- 10 means 10 bits per channel, stored in 2 bytes.
- 16 means 16 bits per channel, stored in 2 bytes.

Returns:

True if successful; False if failed.

Attention:

All MUCam cameras support 8-bit depth format; only some support other bit depth format.

- `int MUCam_getBinningCount (MUCam_Handle camera)`

To get the count of binning supported by the camera.

Parameters:

camera The camera handle.

Returns:

The count of binning.



- `bool MUCam_getBinningList(MUCam_Handle camera, int w, int h)`

To get the image size of each binning. The image sizes are in descending order, the full resolution size is w[0] and h[0].

Parameters:

- | | |
|---------------|---|
| <i>camera</i> | The camera handle. |
| <i>w</i> | The integer array that will receive the image width(pixel) of each binning. Must be large enough to hold all the data. |
| <i>h</i> | The integer array that will receive the image height(pixel) of each binning. Must be large enough to hold all the data. |

Returns:

True if successful; False if failed.

See also:

[MUCam_getBinningCount](#)

- `bool MUCam_getBinningType(MUCam_Handle camera, MUCam_Binning_Type * tl)`

To get the type of each binning.

Parameters:

- | | |
|---------------|---|
| <i>camera</i> | The camera handle. |
| <i>tl</i> | The MUCam_Binning_Type array that will receive the type of each binning. Must be large enough to hold all the data. |

Returns:

True if successful; False if failed.

See also:

[MUCam_getBinningCount](#) [MUCam_Binning_Type](#)



- `bool MUCam_setBinningIndex(MUCam_Handle camera, int idx)`

To set the index of the selected binning.

Parameters:

camera The camera handle.

idx The 0-based index of binning.

Returns:

True if successful; False if failed.

See also:

[MUCam_getBinningCount](#)

- `bool MUCam_setROI(MUCam_Handle camera, int* top, int* left, int* bottom, int* right)`

To set the ROI(region of interest) of the camera. The input coordinates are based on the size of currently exported image, without consideration of current binning, ROI, flip and mirror settings. The top-left corner of currently exported image is (0,0), the bottom-right is (W-1, H-1). If the function returns successfully, the input coordinate parameters will be transformed to be based on camera sensor matrix, and its value might be adjusted.

The ROI image width will be adjusted to align on 4-byte boundary for rendering optimization on Windows.

Parameters:

camera The camera handle.

top The 0-based top coordinate(pixel). After successful return, it will be the coordinate on sensor matrix.

left The 0-based left coordinate(pixel). After successful return, it



will be the coordinate on sensor matrix.

bottom The 0-based bottom coordinate(pixel). After successful return, it will be the coordinate on sensor matrix. The bottom-top+1 equals the width(pixel) of ROI.

right The 0-based right coordinate(pixel). After successful return, it will be the coordinate on sensor matrix. The right-left+1 equals the height(pixel) of ROI.

Returns:

True if successful; False if failed.

See also:

[MUCam_getFrame](#)

- `bool MUCam_setFlip(MUCam\_Handle\_camera, bool b)`

To flip the camera image.

Parameters:

camera The camera handle.

b The flip setting.

Returns:

True if successful; False if failed.

- `bool MUCam_setMirror(MUCam\_Handle\_camera, bool b)`



To mirror the camera image.

Parameters:

camera The camera handle.

b The mirror setting.

Returns:

True if successful; False if failed.

- `int MUCam_getGainCount(MUCam_Handle camera)`

To get the count of the gain supported by camera. The camera gain is a series of discrete values, the application only can select one of them.

Parameters:

camera The camera handle.

Returns:

The count of the gain supported by camera.

- `bool MUCam_getGainList(MUCam_Handle camera, float* g)`

To get the gain value list.

Parameters:

camera The camera setting.

g The float array, must be large enough to hold the data.

Returns:

True if successful; False if failed.

See also:

[MUCam_getGainCount](#)



- `bool MUCam_setRGBGainIndex(MUCam_Handle camera, int r, int g, int b)`

To set the gain index of RGB channel.

Parameters:

camera The camera handle.

r The 0-based gain index of red channel.

g The 0-based gain index of green channel.

b The 0-based gain index of blue channel.

Returns:

True if successful; False if failed.

Attention:

Some cameras don't support different gain values in RGB channels. The function will return true only when the parameters r,g,b are the same.

See also:

[MUCam_getGainCount](#)

- `bool MUCam_setRGBGainValue(MUCam_Handle camera, float r, float g, float b, int* ri, int* gi, int* bi)`

```
void doWhiteBalance()
{
    ROI = getROI();
    for (1 to 3)
    {
        Image = grabImage();
        CalculateWhiteBalance(Image, ROI, &redGain,
                              &greenGain, &blueGain);
    }
}
```



```
if (redGain != 1 && greenGain != 1 && blueGain != 1)
{
    redGain    *= gainValue[currentRedGainIndex];
    greenGain  *= gainValue[currentGreenGainIndex];
    blueGain   *= gainValue[currentBlueGainIndex];

    MUCam_setRGBGainValue(camera, redGain, greenGain,
                           blueGain, &currentRedGainIndex,
                           &currentGreenGainIndex,
                           currentBlueGainIndex);
}
else
{
    break;
}
}
```

To set the gain value of RGB channel. When the application calculated the expected gain value(e.g. do white-balance processing) it could invoke the function to set the gain value directly. The function will find the closest gain value in gain value list, and return the index actual set. Because the camera hardware gain is not an exact multiply calculation, the application could invoke the function several times to get better result. The above-mentioned sample code shows the procedure.

The white-balance of image is influenced by the light source. The application should re-calculate white-balance gain value after changing the light source. But, the exposure time does not affect the white-balance status. It is suggested that the application adjusts exposure time instead of changing the light source.

**Parameters:**

<i>camera</i>	The camera handle.
<i>r</i>	The expected gain value of red channel.
<i>g</i>	The expected gain value of green channel.
<i>B</i>	The expected gain value of blue channel.
<i>ri</i>	The pointer to the buffer that will receive the gain index of red channel. Might be 0 when it is not used.
<i>gi</i>	The pointer to the buffer that will receive the gain index of green channel. Might be 0 when it is not used.
<i>bi</i>	The pointer to the buffer that will receive the gain index of blue channel. Might be 0 when it is not used.

Returns:

True if successful; False if failed.

Attention:

Some cameras don't support different gain values in RGB channels. The function will return true only when the parameters *r,g,b* are the same.

See also:

[MUCam_getGainCount](#), [MUCam_setRGBGainIndex](#)

- `bool MUCam_getOffsetRange(MUCam\_Handle camera, int* min, int* max)`

To get the offset range of the camera.

Attention:

The offset value is not based on gray level. It just means the offset effect that can be achieved by the camera.



Parameters:

<i>camera</i>	The camera handle.
<i>min</i>	The pointer to the buffer that will receive the minimum offset value.
<i>max</i>	The pointer to the buffer that will receive the maximum offset value.

Returns:

True if successful; False if failed.

- bool MUCam_setRGBOffset([MUCam_Handle camera](#), int *r*, int *g*, int *b*)

To set the offset value of RGB channel.

Attention:

The offset value is not based on gray level, e.g. the offset 100 does not mean the actual pixel gray level can be increased 100.

Parameters:

<i>camera</i>	The camera handle.
<i>r</i>	The offset value of red channel.
<i>g</i>	The offset value of green channel.
<i>b</i>	The offset value of blue channel.

Returns:

True if successful; False if failed.

Attention:

Some cameras don't support different offset values in RGB channels.
The function will return true only when the parameters *r*, *g*, *b* are the same.

See also:

[MUCam_getOffsetRange](#)



- `bool MUCam_getExposureRange(MUCam_Handle camera, float * min, float * max)`

To get the current exposure time range of the camera. The setting of binning and ROI will change the exposure time range, the application should have a check after finishing these settings.

Parameters:

- | | |
|---------------|--|
| <i>camera</i> | The camera handle. |
| <i>min</i> | The pointer to the buffer that will receive the minimum exposure value. For MC1002/2002, it's a value without unit; for other camera, it's based on millisecond. |
| <i>max</i> | The pointer to the buffer that will receive the maximum exposure value. For MC1002/2002, it's a value without unit; for other camera, it's based on millisecond. |

Returns:

True if successful; False if failed.

See also:

[MUCam_setBinningIndex](#), [MUCam_setROI](#)

- `bool MUCam_setExposure(MUCam_Handle camera, float t)`

To set the exposure time of the camera.

Attention:

The exposure time should be set after setting binning or ROI.

Parameters:

- | | |
|---------------|--------------------|
| <i>camera</i> | The camera handle. |
| <i>t</i> | The exposure time. |



Returns:

True if successful; False if failed.

See also:

[MUCam_getExposureRange](#), [MUCam_setBinningIndex](#), [MUCam_setROI](#)

- `bool MUCam_isCoolerAvailable(MUCam\_Handle camera)`

To check whether the cooler is available in the camera.

Parameters:

camera The camera handle.

Returns:

True if the camera has a cooler; False if not.

- `bool MUCam_activateCooler(MUCam\_Handle camera a, bool act)`

To activate or inactivate the cooler in the camera.

Parameters:

camera The camera handle.

act True for activating the cooler, false for inactivating.

Returns:

True if successful; False if failed.

See also:

[MUCam_isCoolerAvailable](#)

- `int MUCam_getFrequencyCount(MUCam\_Handle camera)`



To get the working frequency count supported by the camera.

Parameters:

camera The camera handle.

Returns:

The count of working frequency. The available working frequency is from 0 to n-1, 0 means the fastest.

- `int MUCam_getFrequencyIndex(MUCam\_Handle camera)`

To get current working frequency index of the camera.

Parameters:

camera The camera handle.

Returns:

Current working frequency index.

- `bool MUCam_setFrequencyIndex(MUCam\_Handle camera, int level)`

To set the working frequency index of the camera.

Parameters:

camera The camera handle.

level The index of working frequency.

Returns:

True if successful; False if failed.

See also:

[MUCam_getFrequencyCount](#)



- `bool MUCam_setTriggerType(MUCam_Handle camera, MUCam_Trigger_Type type)`

To set the trigger mode of the camera.

Parameters:

camera The camera handle.

type The type of trigger mode.

Returns:

True if successful; False if failed.

See also:

[MUCam_Trigger_Type](#)

- `bool MUCam_getTemperature(MUCam_Handle camera, float *st, float *at, float *rh)`

To get temperature and humidity values. Only some cameras support the function.

Parameters:

camera The camera handle.

st Sensor surface temperature (degrees Celsius)

at Ambient temperature (degrees Celsius)

rh Relative humidity

Returns:

True if successful; False if failed.

- `bool MUCam_isUSB3Cam (MUCam_Handle camera, int*flag)`



To check whether the camera is a usb3.0 camera

Parameters:

camera The camera handle.

flag receive current camera working mode, return 0 means working on USB2.0 mode, 1 means working on USB3.0 mode

Returns:

True if it is USB3.0 camera, otherwise USB2.0 camera.

- `int MUCam_getBayerFormat(MUCam\_Handle camera)`

To get the format of Bayer data (Raw data)

Parameter:

camera The camera handle.

Returns:

Return Bayer format, the return value minus 7 will equate to the Bayer type defined in [MUCam_Format](#)

See also:

[MUCam_getBayer](#)

[MUCam_bayer2RGB](#)

- `bool MUCam_getBayer(MUCam\_Handle camera, unsigned char*buf, unsigned long*ts)`

To grab a frame of bayer image data, the format of image can be obtained from `MUCam_getBayerFormat`, and the bayer image data can be converted to RGB image data by calling `MUCam_bayer2RGB`

Parameters:

camera The camera handle.



- buf* The frame buffer, must be large enough to hold the image data.
- ts* The pointer to the buffer that will receive time stamp(ms) of the frame. Might be 0 when it is not used.

Returns:

True if successful; False if failed.

See also:

[MUCam_getBayerFormat](#)

[MUCam_bayer2RGB](#)

- `bool MUCam_bayer2RGB(MUCam_Handle camera, unsigned char* bayer, int fmt, int w, int h, int bitcount, unsigned char* rgb).`

Convert the bayer image data to RGB image

Parameters:

- camera* The camera handle.
- bayer* The bayer image buffer, read from MUCam_getBayer.
- fmt* The format of bayer, read from MUCam_getBayerFormat
- w* The width of image
- h* The Height of image
- bitcount* The color depth for the image , the default value is 8
- rgb* The RGB image buffer, must be large enough to hold the image data.

Returns:

True if successful; False if failed.

See also:

[MUCam_getBayerFormat](#)

[MUCam_getBayer](#)



- `bool MUCam_bayer2RGB(MUCam_Handle camera, unsigned char* bayer, int fmt, int w, int h, int bitcount, unsigned char* rgb).`

Convert the bayer image data to RGB image

Parameters:

<i>camera</i>	The camera handle.
<i>bayer</i>	The bayer image buffer, read from MUCam_getBayer.
<i>fmt</i>	The format of bayer, read from MUCam_getBayerFormat
<i>w</i>	The width of image
<i>h</i>	The Height of image
<i>bitcount</i>	The color depth for the image , the default value is 8
<i>rgb</i>	The RGB image buffer, must be large enough to hold the image data.

Returns:

True if successful; False if failed.

See also:

[MUCam_getBayerFormat](#)

[MUCam_getBayer](#)

- `bool MUCam_bayer2RGB(MUCam_Handle camera, unsigned char* bayer, int fmt, int w, int h, int bitcount, unsigned char* rgb).`

Convert the bayer image data to RGB image

Parameters:

<i>camera</i>	The camera handle.
<i>bayer</i>	The bayer image buffer, read from MUCam_getBayer.
<i>fmt</i>	The format of bayer, read from MUCam_getBayerFormat
<i>w</i>	The width of image



h The Height of image
bitcount The color depth for the image , the default value is 8
rgb The RGB image buffer, must be large enough to hold the image data.

Returns:

True if successful; False if failed.

See also:

[MUCam_getBayerFormat](#)

[MUCam_getBayer](#)

- `bool MUCam_bayer2RGB(MUCam_Handle camera, unsigned char* bayer, int fmt, int w, int h, int bitcount, unsigned char* rgb).`

Convert the bayer image data to RGB image

Parameters:

camera The camera handle.
bayer The bayer image buffer, read from MUCam_getBayer.
fmt The format of bayer, read from MUCam_getBayerFormat
w The width of image
h The Height of image
bitcount The color depth for the image , the default value is 8
rgb The RGB image buffer, must be large enough to hold the image data.

Returns:

True if successful; False if failed.

See also:

[MUCam_getBayerFormat](#)

[MUCam_getBayer](#)



- `bool MUCam_bayer2RGB(MUCam_Handle camera, unsigned char* bayer, int fmt, int w, int h, int bitcount, unsigned char* rgb).`

Convert the bayer image data to RGB image

Parameters:

<i>camera</i>	The camera handle.
<i>bayer</i>	The bayer image buffer, read from MUCam_getBayer.
<i>fmt</i>	The format of bayer, read from MUCam_getBayerFormat
<i>w</i>	The width of image
<i>h</i>	The Height of image
<i>bitcount</i>	The color depth for the image , the default value is 8
<i>rgb</i>	The RGB image buffer, must be large enough to hold the image data.

Returns:

True if successful; False if failed.

See also:

[MUCam_getBayerFormat](#)

[MUCam_getBayer](#)

- `bool MUCam_setAutoExposure(MUCam_Handle camera, int target, float* exposure)`

set Auto Exposure, Only some cameras support the function(MoticamProS8 Lite,MoticamProS8 Lite M only currently)

Parameters:

<i>camera</i>	The camera handle.
<i>target</i>	Target brightness, range: 0~255, 0 means manual



exposure, 1~255 for auto exposure

exposure Current exposure value, Only available when target=0

Returns:

True if successful; False if the camera does not support this function.

See also:

[MUCam_setAERect](#)

- `bool MUCam_setAERect(MUCam_Handle camera, int x, int y, int w, int h)`

Set the reference area for the auto exposure, Only some cameras support the function(MoticamProS8 Lite, MoticamProS8 Lite M only currently)

Parameters:

camera The camera handle.

camera The camera handle.

x The 0-based left coordinate(pixel).

y The 0-based top coordinate(pixel).

w The width (pixel) of reference area.

h The height(pixel) of reference area.

Returns:

True if successful; False if the camera does not support this function.

See also:

[MUCam_setAutoExposure](#)

- `bool MUCam_setWhiteBalance(MUCam_Handle camera, int status)`



Set the white balance status, Only some cameras support the function(MoticamProS8 Lite,MoticamProS8 Lite M only currently)

Parameters:

<i>camera</i>	The camera handle.
<i>status</i>	The white balance status, 0 close white balance, 1 calculate and apply the white balance base on current image, 2 open white balance by using the last white balance paramters

Returns:

True if successful; False if the camera does not support this function.

See also:

[MUCam_setAWBRect](#)

- `bool MUCam_setAWBRect(MUCam\_Handle camera, int x, int y, int w, int h)`

Set the reference area for the white balance,Only some cameras support the function(MoticamProS8 Lite,MoticamProS8 Lite M only currently)

Parameters:

<i>camera</i>	The camera handle.
<i>camera</i>	The camera handle.
<i>x</i>	The 0-based left coordinate(pixel).
<i>y</i>	The 0-based top coordinate(pixel).
<i>w</i>	The width (pixel) of reference area.
<i>h</i>	The height(pixel) of reference area.

Returns:

True if successful; False if the camera does not support this function.

See also:

[MUCam_setWhiteBalance](#)



Motic China Group Co., Ltd.

© 2002-2008 Motic China Group Co., Ltd. 版权所有。Motic 是 Motic China Group Co., Ltd. 的注册商标。
Microsoft, Windows 以及 Windows 徽标是 Microsoft Corporation 的注册商标。
所有其他产品及品牌均为其持有者的商标或注册商标。

© 2002-2008 Motic China Group Co., Ltd. All rights reserved.
Motic is a registered trademark and service mark of Motic China Group Co., Ltd.
Microsoft, Windows and the Windows logo are registered trademarks of Microsoft Corporation.
All other trademarks are the property of their respective owners.

设计更改：因为技术进步，生产商有权在设计上作出革新，不再另行通知。

Design Change: The manufacturer reserves the right to make changes in instrument design in accordance with scientific and mechanical progress, without notice and without obligation.